

김승환 프론트엔드 포트폴리오

소개

저는 **Next.js** 기반 레거시 전환, **프론트엔드 구조 설계**, **초기 로딩 성능 개선**에 강점이 있는 프론트엔드 개발자입니다.

Java / Spring / JSP 기반 서비스 개발로 커리어를 시작해 백엔드, DB, 운영 환경까지 서비스 흐름을 경험했고, 이후 React와 Next.js 중심의 프론트엔드 개발로 전문성을 확장했습니다. 현재는 메디게이트 서비스의 JSP 기반 레거시 화면을 Next.js 기반 모던 웹 애플리케이션으로 전환하며, 프론트엔드 파트의 구조 설계와 개발 기준 정리를 함께 담당하고 있습니다.

제가 가장 선명하게 기여할 수 있는 영역은 **기존 서비스의 복잡도를 이해하고, React / Next.js / TypeScript 기반으로 유지보수 가능한 화면 구조와 상태 관리 흐름을 만드는 일**입니다. 성능은 Lighthouse, LCP, FCP, 번들 사이즈를 기준으로 확인하고, 컴포넌트 구조는 재사용성과 변경 용이성을 기준으로 설계합니다.

AI 에이전트, 배포 자동화, 개발자 경험 개선도 함께 다루고 있지만, 이 포트폴리오에서는 저의 핵심 역할을 **프론트엔드 개발자**로 두고, DX와 AI 에이전트 경험은 팀 생산성 개선을 위한 보조 강점으로 정리했습니다.

한 줄 포지셔닝

- 1

JSP 기반 레거시 서비스를 **Next.js** 기반 프론트엔드로 전환하고,
- 2

성능 지표와 유지보수성을 기준으로 화면 구조를 개선하는 프론트엔드 개발자입니다.

핵심 역량

영역	핵심 내용	증명 경험
Next.js / React	Next.js 13/14/15 기반 서비스 개발, React Hook, SSR, API Route, Server Action 활용	메디게이트 리뉴얼, 커뮤니티/초빙구직/임대분양 리뉴얼
레거시 마이그레이션	JSP / Spring 기반 화면을 Next.js 기반 구조로 전환	메디게이트 주요 서비스 리뉴얼
상태 관리	React Query, Zustand, Redux Toolkit 기반 서버 상태와 클라이언트 상태 분리	개원입지, 모바일 메디게이트, 리뉴얼 프로젝트
성능 최적화	Lighthouse, LCP/FCP, 번들 분석, 이미지 최적화, 지연 로딩	모바일 메디게이트 로딩 속도 약 50% 감소, 리뉴얼 서비스 초기 로딩 개선
프론트엔드 구조 설계	Turborepo 모노레포, 컴포넌트 모듈화, 공통 UI 구조 설계	커뮤니티 리뉴얼, 개원입지, 메디게이트 리뉴얼
협업과 리뷰	코드 리뷰 프로세스, 개발 컨벤션, 컴포넌트 작성 기준 정리	FE 팀원 3명과 프로젝트 리뷰 기준 운영
백엔드 이해	Java, Spring, JSP, REST API, DB 흐름 이해	API 연동 구조 설계, 인증 흐름, 레거시 분석

기술 스택

구분	기술
Frontend	Next.js, React, TypeScript, JavaScript, TailwindCSS, Chakra UI
State / Data	React Query, Zustand, Redux Toolkit, Axios
Performance	Lighthouse, Web Vitals, Bundle Analyzer, next/image, dynamic import, IntersectionObserver
Architecture	Turborepo, Monorepo, Component Module, SSR, API Route, Server Action
Backend Understanding	Java, Spring Framework, Spring Boot, JSP, REST API, MySQL, Oracle
DevOps / Monitoring	Git, GitHub, Jenkins, Gradle, pm2, Sentry
Collaboration / AI	Jira, Slack, Figma, Claude Code, Codex, AGENTS.md, Skill, Harness
Mobile	React Native, WebView

경력 요약

(주)메디씨앤씨

2019.07 - 재직 중 / 프론트엔드 파트리더

- JSP 기반 레거시 서비스를 Next.js 기반 프론트엔드로 전환
- Next.js 14/15, TypeScript, React Query, Zustand, Turborepo 기반 프론트엔드 구조 설계
- 프론트엔드 파트 기술 방향성, 코드 리뷰 기준, 컴포넌트 작성 기준 정리
- Lighthouse, LCP/FCP, 번들 사이즈 기준으로 초기 로딩 성능 개선
- Sentry, Bundle Analyzer를 활용한 오류 추적 및 성능 개선 기반 구축
- Java / Spring / JSP 기반 레거시 구조를 이해하고 API 연동, 인증, 화면 전환 흐름을 함께 고려

(주)젠탐테크놀로지

2015.03 - 2019.03 / Java·Spring 기반 풀스택 개발자

- Spring MVC, Spring Boot, JSP 기반 웹 시스템 개발
- REST API 설계 및 구현
- MySQL, Oracle 기반 DB 설계 및 쿼리 작성
- 관리자 페이지, 예약 관리, 관제 시스템, 기록 관리 시스템 개발
- PHP, JSP 기반 레거시 시스템을 Spring Boot 기반 구조로 전환
- React와 Spring Boot를 활용한 프론트엔드/API 연동 경험 확보

대표 프로젝트

메디게이트 서비스 리뉴얼 및 고도화

2025.11 - 진행 중 / 프론트엔드 리드 개발자

프로젝트 성격

JSP 기반으로 운영되던 메디게이트 주요 서비스를 Next.js 기반 모던 웹 애플리케이션으로 전환하고, 이후 Next.js 15 업그레이드, Sentry 도입, 번들 분석, 성능 개선을 진행한 프로젝트입니다.

문제 상황

- JSP 기반 화면과 스크립트가 서비스별로 분산되어 유지보수 비용이 높았습니다.
- 공통 UI와 상태 관리 기준이 부족해 프로젝트마다 구현 방식이 달라질 수 있었습니다.
- 오류 발생 시 원인 추적이 느리고, 초기 로딩 성능을 정량적으로 확인하는 흐름이 부족했습니다.

기술 판단

판단	선택	이유
프레임워크	Next.js 14/15	라우팅, SSR, 이미지 최적화, 코드 스플리팅, 운영 구조를 프레임워크 차원에서 관리하기 위해 선택
언어	TypeScript	API 응답, 컴포넌트 Props, 상태 타입을 명확히 해 변경 시 오류를 줄이기 위해 선택
서버 상태	React Query	API 요청, 캐싱, 로딩/에러 상태, refetch 흐름을 일관되게 관리하기 위해 선택
클라이언트 상태	Zustand	모달, 필터, 선택값 등 UI 상태를 가볍게 분리하기 위해 선택
모노레포	Turborepo	서비스별 공통 설정과 컴포넌트 재사용 구조를 만들기 위해 선택
모니터링	Sentry	운영 중 오류 원인과 발생 맥락을 더 빠르게 파악하기 위해 선택

주요 기여

- JSP 기반 레거시 화면을 Next.js 기반 프론트엔드 구조로 전환
- Next.js 14에서 15로의 업그레이드 방향 검토 및 적용
- React Query와 Zustand 기반 상태 관리 기준 수립
- Turborepo 기반 모노레포 구조 설계 및 구축
- JWT 인증 관리와 API 연동 흐름 정리
- Sentry 도입을 위한 요구사항 조사 및 적용 방향 수립
- Bundle Analyzer 기반 의존성 분석 및 불필요한 번들 정리
- 코드 리뷰 프로세스와 컴포넌트 모듈화 기준 정리

성과와 측정 기준

- Lighthouse, LCP, FCP 기준으로 초기 로딩 성능을 확인하는 흐름을 마련했습니다.
- Bundle Analyzer로 큰 의존성과 불필요한 import를 확인하고 정리했습니다.
- Sentry 도입으로 오류 발생 위치와 사용자 환경을 추적할 수 있는 기반을 만들었습니다.
- 프로젝트별로 흩어져 있던 컴포넌트 작성 기준을 정리해 리뷰 기준을 맞췄습니다.

결과물 증거

- Next.js 기반 서비스 전환 구조
- Turborepo 기반 모노레포 구조
- Sentry 오류 모니터링 흐름
- Bundle Analyzer 기반 번들 분석 결과
- 코드 리뷰 기준과 컴포넌트 모듈화 기준

임대분양 서비스 리뉴얼

2025.06 - 2025.11 / 프론트엔드 리드 개발자

프로젝트 성격

기존 JSP 기반 임대분양 서비스를 Next.js 기반 모던 웹 애플리케이션으로 전환한 프로젝트입니다. 공통 컴포넌트, 결제 모듈, 이미지 최적화, 지연 로딩 전략을 정리하며 초기 로딩 속도와 유지보수성을 개선했습니다.

문제 상황

- JSP 기반 화면 구조로 인해 화면 변경 시 영향 범위 파악이 어려웠습니다.
- 이미지와 목록 콘텐츠가 많은 화면에서 초기 로딩 부담이 있었습니다.
- 결제 흐름과 광고 화면이 함께 얹혀 있어 컴포넌트 분리가 필요했습니다.

기술 판단

- Next.js를 사용해 라우팅, 페이지 단위 구조, 코드 스플리팅을 일관되게 관리했습니다.
- `next/image` 를 사용해 이미지 최적화 기준을 만들었습니다.
- IntersectionObserver 기반 지연 로딩을 적용해 초기 화면에 필요하지 않은 리소스 로드를 늦췄습니다.
- React Query와 Zustand를 분리해 서버 데이터와 UI 상태가 섞이지 않도록 했습니다.

주요 기여

- 프론트엔드 아키텍처 설계 주도
- 레거시 서비스 전환 가이드 제공
- 재사용 가능한 컴포넌트 구조 설계
- 결제 모듈 작성
- `next/image` 기반 이미지 최적화
- IntersectionObserver 기반 지연 로딩
- 코드 리뷰 및 개발 기준 정리

성과와 측정 기준

- 기존 대비 페이지 초기 로딩 속도 20~30% 개선
- 측정 기준: 동일 주요 페이지를 기준으로 리뉴얼 전후 초기 로딩 지표 비교
- 공통 컴포넌트 구조 정리로 유사 화면 개발 시 재사용 가능한 단위를 확보
- 레거시 전환 과정의 개발 기준 정립

결과물 증거

- 리뉴얼 전후 주요 페이지 로딩 지표
- 결제 모듈
- 이미지 최적화 및 지연 로딩 적용 화면
- 재사용 가능한 공통 컴포넌트 구조

초빙구직 서비스 리뉴얼

2025.02 - 2025.05 / 프론트엔드 리드 개발자

프로젝트 성격

JSP 기반 초빙구직 서비스를 Next.js 기반 모던 웹 애플리케이션으로 전환한 프로젝트입니다. 공통 컴포넌트, 결제 모듈, ErrorBoundary, 이미지 최적화 전략을 적용했습니다.

문제 상황

- 구직/공고 화면의 상태와 결제 흐름이 복잡해 화면별 예외 처리가 필요했습니다.
- 장애나 API 오류 발생 시 사용자에게 안정적인 피드백을 주는 구조가 부족했습니다.
- 공통 컴포넌트 기준이 없으면 유사 화면이 중복 구현될 가능성이 있었습니다.

기술 판단

- ErrorBoundary를 적용해 일부 화면 오류가 전체 서비스 경험으로 번지지 않도록 했습니다.
- React Query를 사용해 API 요청 상태와 에러 상태를 화면에서 일관되게 처리했습니다.
- 공통 컴포넌트 단위를 먼저 정의해 이후 유사 화면 구현의 기준으로 사용했습니다.

주요 기여

- 프로젝트 리딩 및 기술 방향성 결정
- 컴포넌트 모듈화 전략 수립
- ErrorBoundary 도입을 통한 사용자 경험 개선
- 레거시 서비스 전환 가이드 제공
- 결제 모듈 작성
- `next/image` 기반 이미지 최적화
- IntersectionObserver 기반 지연 로딩

성과와 측정 기준

- 기존 대비 페이지 초기 로딩 속도 20~30% 개선
- 측정 기준: 주요 진입 페이지의 리뉴얼 전후 초기 로딩 지표 비교
- 장애 상황에서 전체 화면이 깨지지 않도록 ErrorBoundary 기반 예외 처리 구조 마련
- 공통 컴포넌트 기반 개발 생산성 개선

결과물 증거

- ErrorBoundary 적용 화면
- 결제 모듈
- 공통 컴포넌트 구조
- 리뉴얼 전후 초기 로딩 비교 지표

커뮤니티 서비스 리뉴얼

2024.10 - 2025.01 / 프론트엔드 리드 개발자

프로젝트 성격

기존 JSP 기반 커뮤니티 서비스를 Next.js 기반 모던 웹 애플리케이션으로 전환한 마이그레이션 프로젝트입니다. 모노레포 구조, 인증 관리, 컴포넌트 모듈화, SSR 기반 사용자 경험 개선을 중심으로 진행했습니다.

문제 상황

- 커뮤니티 화면은 목록, 상세, 댓글, 인증 상태 등 상태 변화가 많았습니다.
- JSP 기반 구조에서는 화면과 서버 로직의 경계가 명확하지 않아 변경 영향 파악이 어려웠습니다.
- 로그인 상태와 API 연동 흐름을 프론트엔드에서 일관되게 관리할 필요가 있었습니다.

기술 판단

- Turborepo 기반 모노레포를 도입해 공통 설정과 컴포넌트를 분리했습니다.
- JWT 인증 흐름을 정리해 API 연동과 화면 접근 제어 기준을 맞췄습니다.
- API Route와 Server Action 활용 가능성을 검토해 서버 데이터 처리 구조를 정리했습니다.

주요 기여

- Turborepo 기반 모노레포 구조 설계 및 구축
- 프론트엔드 아키텍처 설계
- 컴포넌트 모듈화 전략 수립
- JWT 인증 관리
- API Route와 Server Action을 활용한 서버 데이터 처리 구조 설계
- 코드 리뷰 프로세스 수립

성과와 측정 기준

- 기존 대비 페이지 로딩 속도 30% 개선
- Lighthouse 기준 LCP 및 FCP 개선
- SSR 기반 사용자 경험 개선
- 컴포넌트 모듈화로 개발 생산성 향상

결과물 증거

- Turborepo 모노레포 구조
- JWT 인증 처리 흐름
- 커뮤니티 목록/상세 화면 전환 구조
- Lighthouse 전후 비교 지표

메디게이트 개원입지 서비스 개발

2023.06 - 2024.05 / 프론트엔드 리드 개발자

프로젝트 성격

네이버 지도 API를 사용한 Next.js 기반 개원 위치 분석 서비스입니다. 지도 기반 UI와 차트 기반 데이터 시각화를 함께 구현하고, 자사 및 타사 데이터를 활용한 보고서 기능을 제공했습니다.

문제 상황

- 지도, 차트, 보고서 UI가 한 화면에 함께 존재해 렌더링 비용과 상태 관리 복잡도가 높았습니다.
- 지도 API, 자사 데이터, 타사 데이터가 함께 사용되어 데이터 흐름을 명확하게 분리해야 했습니다.
- 사용자가 지도 조작과 보고서 확인을 함께 수행하므로 UI 반응성이 중요했습니다.

기술 판단

- React Query로 외부/내부 API 데이터를 관리하고, Zustand로 지도 UI 선택 상태를 분리했습니다.
- Recharts를 사용해 보고서성 데이터를 시각적으로 확인할 수 있게 했습니다.
- 지도 UI와 보고서 화면을 분리해 불필요한 리렌더링을 줄이는 방향으로 컴포넌트를 설계했습니다.

주요 기여

- 프로젝트 전반의 기술 방향성 결정
- Turborepo 기반 모노레포 구조 설계 및 구축
- 컴포넌트 모듈화 전략 수립
- 네이버 지도 API 사용 전략 수립
- 지도 UI와 보고서 화면의 상태 관리 구조 설계
- Recharts 기반 데이터 시각화 화면 구현

성과와 측정 기준

- 메모이제이션 최적화로 불필요한 리렌더링 감소
- 모노레포 도입으로 코드 재사용성 증가
- 컴포넌트 모듈화로 개발 생산성 향상
- 지도 및 차트 기반 UI 구현 경험 확보

결과물 증거

- 네이버 지도 기반 분석 화면
- Recharts 기반 보고서 화면
- 지도 선택 상태와 보고서 데이터 상태 분리 구조
- 메모이제이션 적용 컴포넌트

메디게이트 모바일 서비스 개발

2022.10 - 2023.04 / 프론트엔드 개발자

프로젝트 성격

Next.js 기반 모바일 메디게이트 서비스입니다. Next.js와 TypeScript 도입을 위한 사전 스터디와 팀 내 교육을 진행하고, 서버 상태와 클라이언트 상태를 분리하는 구조를 제안했습니다.

문제 상황

- 모바일 환경에서 초기 로딩 속도와 정적 리소스 관리가 중요했습니다.
- 기존 구조에서는 API 요청 상태와 UI 상태가 섞이기 쉬웠습니다.
- 팀 내 Next.js와 TypeScript 사용 경험을 맞추는 과정이 필요했습니다.

기술 판단

- Next.js를 도입해 페이지 구조와 초기 로딩 최적화 가능성을 확보했습니다.
- TypeScript를 도입해 컴포넌트 Props와 API 응답 타입을 명확히 했습니다.
- React Query와 Redux Toolkit을 함께 사용해 서버 상태와 클라이언트 상태를 분리했습니다.

- CDN 캐싱 전략을 적용해 정적 리소스 로딩 부담을 줄였습니다.

주요 기여

- 개발 컨벤션 논의 주도
- Next.js 도입을 위한 사전 스터디 및 교육
- TypeScript 도입을 위한 사전 스터디 및 교육
- React Query 및 Redux Toolkit 도입 주도
- 컴포넌트 모듈화 전략 수립
- 정적 리소스 CDN 처리 및 캐싱 전략 적용

성과와 측정 기준

- 기존 모바일 서비스 대비 로딩 속도 약 50% 감소
- Lighthouse 기준 LCP 및 FCP 개선
- TypeScript 도입으로 런타임 에러율 감소
- 서버 상태와 클라이언트 상태 분리
- 불필요한 API 호출량 감소

결과물 증거

- 모바일 메디게이트 Next.js 화면
- Lighthouse 전후 비교 지표
- CDN 캐싱 적용 정적 리소스 구조
- React Query / Redux Toolkit 상태 분리 구조

프론트엔드 리딩

역할 정의

제 리딩 경험은 관리 직무라기보다, **프론트엔드 개발자가 같은 기준으로 개발할 수 있도록 구조와 기준을 정리한 경험**에 가깝습니다.

- FE 팀원 3명과 함께 프로젝트 진행
- 프로젝트 기술 스택 검토 및 도입 기준 정리
- Next.js, TypeScript, React Query, Zustand 기반 아키텍처 수립
- Turborepo 기반 모노레포 구조 설계
- 코드 리뷰 프로세스 수립 및 리뷰 문화 정착
- 프로젝트별 컴포넌트 모듈화 전략 수립

리뷰에서 주로 확인한 기준

- 컴포넌트가 특정 화면에 과하게 종속되어 있지 않은가
- 서버 상태와 클라이언트 상태가 섞여 있지 않은가
- API 요청의 loading, error, empty 상태가 화면에 반영되어 있는가
- 재사용 가능한 UI와 비즈니스 로직이 분리되어 있는가
- 초기 로딩에 불필요한 리소스를 포함하고 있지 않은가

- 타입 정의가 컴포넌트 사용 의도를 충분히 설명하는가

개발자 경험(DX) 및 AI 에이전트 활용

이 영역은 제 핵심 포지션을 대체하는 경험이라, **프론트엔드 팀의 반복 업무와 코드 품질을 개선하기 위한 보조 강점**입니다.

개발 및 운영 환경 개선

- pm2 기반 무중단 배포 환경 구축
- Jenkins 파이프라인 구성
- Groovy 스크립트를 이용한 배포 파이프라인 작성
- Git 도입 및 팀 내 사용 가이드 교육
- 브랜치 관리 전략 수립
- Slack, Jira 기반 협업 프로세스 정리
- 신규 입사자 온보딩 프로세스 구축

AI 에이전트 기반 팀 생산성 개선

프론트엔드 개발 생산성과 코드 품질을 높이기 위해 Claude Code / Codex 기반 AI 에이전트 활용 방식을 팀에 공유하고 있습니다.

- 업무별 스킬 5종 이상 작성 및 공유
- 반복 업무 자동화를 위한 하네스 2종 작성 및 공유
- 개발팀 11명을 대상으로 AI 에이전트 활용 관련 발표 3회 진행
- 하네스 개념과 활용 방식에 대한 팀 내 발표 진행
- AGENTS.md 작성 및 프로젝트별 에이전트 작업 가이드 공유
- 반복되는 코드 분석, 리팩토링 준비, 문서화, 작업 가이드 작성을 재사용 가능한 흐름으로 정리

Java / Spring 기반 경험

Java / Spring 기반 서비스 리뉴얼

2020.02 - 2022.05 / 풀스택 개발 및 프론트엔드 개발

- Spring, JSP 기반 서비스 리뉴얼
- 레거시 코드 분석 및 리팩토링
- 스크립틀릿 제거 및 코드 최적화
- 파일 구조 최적화 및 중복 코드 제거
- 쿼리 최적화를 통한 성능 개선
- Kakao Map API, 공공데이터 API 연동
- React Native 기반 모바일 앱 및 WebView 최적화

주요 성과

- 초빙정보 서비스 페이지 로딩 속도 약 40% 개선
- 커뮤니티 서비스 목록 로딩 속도 15% 개선
- 병원 임대/분양 서비스 MAU 10% 증가

- React Native WebView 기반 모바일 앱 최적화 경험 확보

초기 SI 및 시스템 개발 경험

2015.03 - 2019.03 / Java·Spring 기반 풀스택 개발

- Spring MVC, Spring Boot 기반 웹 시스템 개발
- MySQL, Oracle 기반 DB 설계 및 쿼리 작성
- REST API 설계 및 구현
- 관리자 페이지 개발
- 실시간 센서 데이터 처리
- Ajax 기반 비동기 통신 구현
- Gradle 빌드 시스템 도입
- PHP 시스템의 Spring Boot 마이그레이션
- React 기반 프론트엔드와 Spring Boot REST API 서버 개발 경험

일하는 방식

문제를 먼저 정의합니다

기술을 먼저 고르기보다, 화면 변경이 어려운 이유, 성능이 느린 지점, 상태가 복잡해지는 원인을 먼저 확인합니다. 이후 프레임워크, 상태 관리, 컴포넌트 구조, 성능 개선 방식을 결정합니다.

성능은 수치로 확인합니다

Lighthouse, LCP, FCP, 번들 사이즈, 렌더링 비용을 기준으로 사용자 경험을 확인합니다. 성능 개선 결과는 가능한 한 리뉴얼 전후 비교, 동일 페이지 기준, 동일 측정 도구 기준으로 남기려고 합니다.

유지보수 가능한 구조를 우선합니다

한 번의 기능 구현보다 팀이 반복해서 사용할 수 있는 컴포넌트, 컨벤션, 가이드, 리뷰 기준을 중요하게 생각합니다. 모노레포, 타입 안정성, 상태 관리 분리, 컴포넌트 모듈화를 통해 장기적으로 유지보수 가능한 프론트엔드 구조를 지향합니다.

관심사

- React 19 / Next.js 15
- Server Components / Server Actions
- 캐싱 전략
- Turborepo 기반 모노레포 안정화
- 웹 성능 최적화
- 접근성과 사용자 경험 개선
- AI 에이전트 기반 프론트엔드 개발 워크플로
- 코드 리뷰 문화 개선